

6

Set It in Motion! – Animating and Moving around Your World

Our first game, while functional, certainly wasn't that visually appealing, at least not for this century. First of all, the graphics barely represented what they were supposed to. Referring to a chain of blocks as a snake was the only thing that gave the player an idea of what they were in control of. The second staple of an older design was the static camera position. While it is a design choice in a game like *Snake*, more complex genres would be crippled by such a limitation. Titles like *Super Mario Bros* rely on the fact that the game world extends beyond the boundaries of your screen, not only because of visual appeal, but also because of the ability to build a larger game world that doesn't have to fit within a certain pre-designated rectangle. A simple decision to represent game characters with images instead of basic shapes, as well as providing the means for the screen to be moved opens up a lot of doors.

In this chapter, we will be covering:

- SFML views and screen scrolling
- Automated resource management and handling
- Creation and application of sprite-sheets
- Sprite-sheet animation

There's a lot to learn, so let's not waste any time and dive right in!

Use of copyrighted resources

Before proceeding any further, we would like to give credit where credit's due to the artists who created the textures, sprites, and other art we're going to be using for our game. These assets include:

- *Lemcraft* by richtaur under CC0 1.0 license: <http://opengameart.org/content/lemcraft>
- *Prototyping 2D Pixelart Tilesets* by <http://www.robotality.com> under CC-BY-SA 3.0 license: <http://opengameart.org/content/prototyping-2d-pixelart-tilesets>
- *Generic Platformer Tileset (16x16) + Background* by etqws3 under CC0 1.0 license: <http://opengameart.org/content/generic-platformer-tileset-16x16-background>
- *Knight and Rat* sprites by backyardninja: http://www.dumbmanex.com/bynd_freestuff.html

The licensing of all the resources listed above allows for any use of the material, even commercial. For more information on the two specific licenses, please visit the following links:

- <http://creativecommons.org/publicdomain/zero/1.0/>
- <http://creativecommons.org/licenses/by-sa/3.0/>

Finding and using the current directory

There's no doubt that after programming for a while, the inconveniences of reading or writing to files start to quickly build up. It may not be so bad when running programs outside of your compiler, but using a relative path can be a pain while debugging, because it is no longer relative to the directory where the executable is, but instead where the `.obj` files are located. For the rest of this book, we will be using a function to obtain the full path to your executable, regardless of where it is or how it's being launched. Let's take a look at a new header file that will contain this function, called `Utilities.h`:

```
#pragma once
#define RUNNING_WINDOWS
#include <iostream>
#include <string>
#include <algorithm>

namespace Utils{
    #ifdef RUNNING_WINDOWS
```

```

#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <Shlwapi.h>
inline std::string GetWorkingDirectory(){
    HMODULE hModule = GetModuleHandle(nullptr);
    if(hModule){
        char path[256];
        GetModuleFileName(hModule,path,sizeof(path));
        PathRemoveFileSpec(path);
        strcat_s(path,"\\"); // new
        return std::string(path); // new
    }
    return "";
}
#elif defined RUNNING_LINUX
#include <unistd.h>
inline std::string GetWorkingDirectory(){
    char cwd[1024];
    if(getcwd(cwd, sizeof(cwd)) != nullptr){
        return std::string(cwd) + std::string("/");
    }
    return "";
}
#endif
}

```

 `#pragma once` is widely supported, but is non-standard. It can be replaced by a typical inclusion guard if the code is being processed in an older compiler.

If a `RUNNING_WINDOWS` macro is set, it defines a method that first obtains the full path including the executable and its extension, then obtains only the name and extension of the executable and finally strips the full path of it before returning the string, that now contains the full "address" of the directory the executable resides in. These functions are specific to Windows and won't work on other operating systems, so this header needs to define the same method differently for each of them.

 Using these functions to obtain the current directory requires the `Shlwapi.h` header file to be included, as well as the `shlwapi.lib` file being listed in the linker's additional dependencies in *all* configurations. Forgetting to fulfill these requirements will cause linker errors.

As you can see, we have Windows and Linux operating systems covered here. It is up to you to implement versions of the same function for other platforms if you want your application to run properly.

Using the SFML views

Up until this point, we have only dealt with code that renders things within the boundaries of the window that's open. There hasn't been an instance where we needed the screen to move yet, which would be fine if we lived in the early days of the 80s, but games even a decade after that time were a lot more advanced. Take, for example, *Super Mario Brothers*, a classic *side-scroller*. Its genre alone pinpoints what our first game didn't have: scrolling. If the scrolling effect or any kind of movement, resizing or rotation of the screen is desirable, using the `sf::View` is necessary.

What is `sf::View`? It's a rectangle. That's it. If you have ever held your fingers in a rectangle shape to "frame" the world you're observing, you have created a view with your hands. By moving it, you are essentially moving through the scene beyond the cut-off point of the window. If you're still not "getting the picture," here's an illustration to lead you in the right direction:



The `sf::View` is a very lightweight object that essentially holds a few floating point variables and a number of methods to retrieve its values. Its constructor can take in a `sf::FloatRect` type, which defines a rectangular area of the screen, or it can take two `sf::Vector2f` types, the first one being the centre of the view and the second one being the size:

```
// Top-left corner at 500:500, bottom-right at 1000:1000.
sf::View first(sf::FloatRect(500,500,1000,1000));
// Center at 250:250, size is 800:600.
sf::View second(sf::Vector2f(250,250), sf::Vector2f(800,600));
```

As you can see, views are mostly manipulated by their centre instead of their top-left corner, unlike most of the other shapes.

Moving the view can be done by manipulating its centre, like so:

```
// Top-left corner at 0:0, bottom-right at 800:600.
sf::View view(sf::FloatRect(0,0,800,600));
view.setCenter(100,100); // Move center to 100:100.
```

It can also be moved by an offset, via the `move` method:

```
view.move(100,100); // Move by 100x100 offset.
```

Resizing the view can be accomplished by either using the `setSize` method or zooming by a factor, using the `zoom` method:

```
view.setSize(640, 480); // Creates a smaller view space.
view.zoom(0.5f); // Also creates a smaller view space.
```

In the first case, it resizes a view of 800x600 px down to 640x480 px. The second case applies a factor of 0.5f to its current size, which cuts it down in half, making everything on screen larger.

In order to use a specific view, you must call the `setView` method of the window instance you're using:

```
window.setView(view); // Applies view to window.
```



The `setView` method does *not* take values in by reference. It simply copies the values from the view and stores them in the window object. If the view is altered at any point in your code, you must call the `setView` method *again* in order to apply those changes and make them reflect.

One last thing worthy of mentioning is that two views can be obtained from the window object as well. The first kind of view is the current one being used, and the second view is the default a window starts with. It's the same size as the window and its left corner is positioned at coordinates (0;0). Retrieving these views can be done like so:

```
sf::View view = window.getView();
sf::View defaultView = window.getDefaultView();
```

Preparing application states for views

In order to add support for panning around our world, we must adjust the state system. The main idea here is that each state has its own view. Something like the main menu will most likely never need to move past the default view the window provides, unlike the game state, which will be focusing on the character every frame. A simple and elegant approach would be to store a view with each state, so it can adjust the screen view space if it needs to. Let's start by modifying the `BaseState.h` file:

```
class BaseState{
public:
    ...
    sf::View& GetView(){ return m_view; }
protected:
    ...
    sf::View m_view;
};
```

Like most of the classes we're going to be modifying, we're only showing the relevant parts here, which are fairly conservative size wise in this case. All we've added is a view data member and a method for obtaining it. Let's move on to putting this view to work in our state manager:

```
void StateManager::CreateState(const StateType& l_type) {
    ...
    BaseState* state = newState->second();
    state->m_view = m_shared->m_wind->
        GetRenderWindow()->getDefaultView();
    ...
}
```

Because we don't want the default constructor of `sf::View` to initialize our view member to its own default values, it's necessary to set the view up when creating the state. Most of the states we have so far rely on the fact that the view never moves, which is why it's set to default first. If a state wishes to define its own view, it can always do so in the `OnCreate` method, as you will see shortly. Let's move on to state switching:

```
void StateManager::SwitchTo(const StateType& l_type) {
    ...
    for(...)
    {
        if(itr->first == l_type) {
            ...
        }
    }
}
```

```

        m_shared->m_wind->GetRenderWindow() ->
            setView(tmp_state->GetView());
        return;
    }
}
...
m_states.back().second->Activate();
m_shared->m_wind->GetRenderWindow() ->setView(
    m_states.back().second->GetView());
}

```

This is fairly straightforward. When switching to a different state, we want to change the view space of the window to match the state we're switching to. If that's not accomplished and the game state moves the view, switching to another state will simply leave you with a blank screen, because the new state's contents are rendered outside the window's view space.

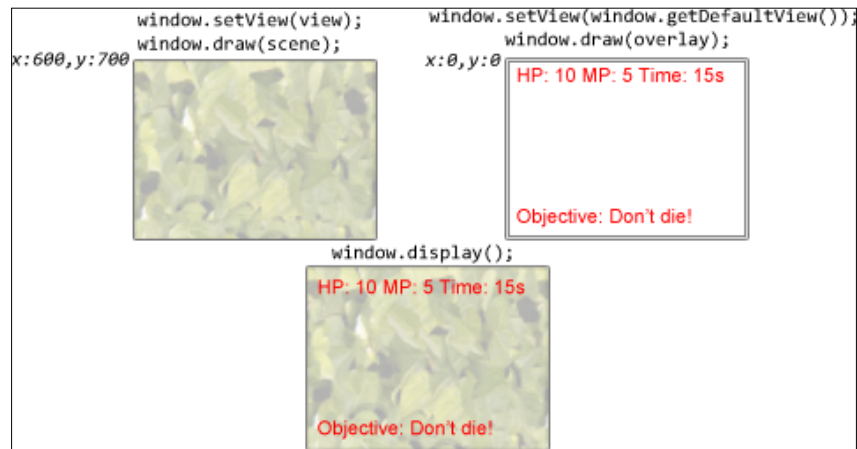
Drawing multiple states at once can raise some problems, now that different views are introduced. This can be a little bit difficult to understand, so let's illustrate the problem with an example. Let's say the game is paused. Because the paused state is transparent, it needs to draw the state before it first, in order to blend them together. Instantly there's a problem, because the paused state positions its elements in window coordinates and it never needs the view to move. If the view of the window does move, whatever the state on top draws is going to be outside its view space and will therefore be either partially visible or not there at all. We could translate window coordinates to world coordinates and update positions of these elements each frame to "follow" the screen, but that's not an elegant or efficient solution. Therefore, we must set the window view to the state view before it gets rendered, like so:

```

void StateManager::Draw() {
    ...
    for(; itr != m_states.end(); ++itr) {
        m_shared->m_wind->GetRenderWindow() ->
            setView(itr->second->GetView());
        itr->second->Draw();
    }
    ...
}

```

Because of the way views and rendering works, the problem above is resolved. Consider the following illustration:



First, the window view is set to the view of the game in order to render the scene. Let's say it's top-left corner is at position (600;700). The default view of the window is then applied. This moves the window view space's top-left corner back to (0;0), which matches the local window coordinates. Because the elements that are about to be drawn are positioned based on these coordinates, they are now in the window's view space again and are drawn on top of the frame buffer. The transparent parts are blended, while opaque pixels are overwritten. Finally, `window.display()` ; gets called and the frame buffer is drawn on screen. The result is both the scene and the elements being blended together.

The last thing we want to add to our existing code-base before moving on is a new method in the Window class for obtaining a `sf::FloatRect` type that defines the window view space:

```
sf::FloatRect Window::GetViewSpace() {  
    sf::Vector2f viewCenter = m_window.getView().getCenter();  
    sf::Vector2f viewSize = m_window.getView().getSize();  
    sf::Vector2f viewSizeHalf(viewSize.x / 2, viewSize.y / 2);  
    sf::FloatRect viewSpace(viewCenter - viewSizeHalf, viewSize);  
    return viewSpace;  
}
```

First, this method obtains the centre and the size of the current view. Then it calculates what half of the size is and subtracts it from the coordinates of the view's centre, in order to obtain its top-left corner. Lastly, it constructs the rectangle of the view space by passing in the top-left corner of the view and its size. This will be useful later on.

Automated resource management

Let's talk about textures and the way we've been using them so far. A texture in SFML is something you want to only have one of, since it's not cheap memory wise. Our approach so far was simply storing the textures as data members of relevant classes. A scenario that illustrates how horrendous this strategy is would be as follows: you need the same texture somewhere else. That's it. It really doesn't seem like the type of thing that you could just brush off your shoulders, as it only happens once in a blue moon. Creating multiple textures that all hold the same data is a huge waste of resources, and adding methods for obtaining textures from the classes that use them is a disaster. Not only does it clutter the class footprint, it would also mean that other classes would have to have access to the one that holds this texture. Nobody should subject themselves to such torture.

How do we remedy this situation then? By creating a class that holds all of our textures in the same place and keeps track of how many times they're used in order to manage its resources in a smart way. We also want to give them unique identifiers in order to refer to them quickly, which can be done by loading them from a file that maps a name to a path. We can name it `Textures.cfg` and it would look something like this:

```
Intro media/Textures/intro.png
PlayerSprite media/Textures/PlayerSheet.png
RatSprite media/Textures/RatSheet.png
TileSheet media/Textures/tilesheet.png
Bg1 media/Textures/bg1.png
Bg2 media/Textures/bg2.png
Bg3 media/Textures/bg3.png
```

This approach, of course, can be used for other types of resources, not just textures. Later on, we'll be working with fonts and sound files as well, so let's design an abstract base class that will take care of all the common tasks first, before delving into handling textures specifically.

Designing a resource manager

All of the resources we're going to be working with are going to be counted and accounted for, so to speak. Whenever we want to use a texture, for example, it needs to be requested. If it's no longer needed, the resource is released. It sounds simple enough, so let's write it:

```
template<typename Derived, typename T>
class ResourceManager{
public:
```

```
    ResourceManager(const std::string& l_pathsFile) {
        LoadPaths(l_pathsFile);
    }

    virtual ~ResourceManager() { PurgeResources(); }
    ...
private:
    std::unordered_map<std::string,
        std::pair<T*, unsigned int>> m_resources;
    std::unordered_map<std::string, std::string> m_paths;
};
```

When dealing with classes that use templates, it's necessary for the implementation of the methods to be in the header file, because the compiler needs to have access to the implementation in order to instantiate the methods with the template arguments. With that being said, let's talk about the `m_resources` data member. It's using a map, which is going to tie a string handle to a pair of elements, the first of which is the template parameter of a resource and the second is an unsigned integer type that will be used as a counter for how many places are currently using this particular resource.

Another data member we have is a map of two strings for the resource paths. The constructor calls an internal method in order to load the paths from a specific location, while the destructor invokes another internal method to purge and de-allocate all of its resources.

Let's begin implementing other methods we're going to need to use this class, starting with the public ones:

```
T* GetResource(const std::string& l_id) {
    auto res = Find(l_id);
    return(res ? res->first : nullptr);
}
```

This is a method for acquiring a resource that's being kept in the manager. It uses the string argument as a handle and looks for it in the map, using the internal `Find` method that we'll define later. It returns a pointer to the pair element of the map if something is found, or `nullptr` if nothing is found.

We also might be interested in retrieving one of the paths to a particular resource:

```
std::string GetPath(const std::string& l_id) {
    auto path = m_paths.find(l_id);
    return(path != m_paths.end() ? path->second : "");
}
```

This system would be useless without some way to guarantee that a resource doesn't get de-allocated while it's still being used. In order to prevent that, let's implement a method that will register the use of a resource:

```
bool RequireResource(const std::string& l_id){
    auto res = Find(l_id);
    if(res){
        ++res->second;
        return true;
    }
    auto path = m_paths.find(l_id);
    if (path == m_paths.end()){ return false; }
    T* resource = Load(path->second);
    if (!resource){ return false; }
    m_resources.emplace(l_id, std::make_pair(resource, 1));
    return true;
}
```

This method serves two purposes. One is simply incrementing the counter of instances a resource is being used when it's required. The second purpose of it is to create a resource, if the handle is not found in the resource container. It first looks as if the resource handle is valid, by checking the path container. If a match is found, it tries to acquire a pointer to newly allocated memory, which gets returned by the `Load` method. If it hasn't returned a `nullptr` value, the resource gets inserted with a counter set to 1.

Just like for every yin there must be a yang, for every resource required, there must be a point where it's no longer needed:

```
bool ReleaseResource(const std::string& l_id){
    auto res = Find(l_id);
    if (!res){ return false; }
    --res->second;
    if (!res->second){ Unload(l_id); }
    return true;
}
```

This method tries to find a resource in the container using the string handle. If one is found, its use counter is decremented. If the counter is now at 0, this resource is no longer needed and the memory for it can be de-allocated by calling the `Unload` method.

At some point, everything must go. This is the purge method:

```
void PurgeResources() {
    while(m_resources.begin() != m_resources.end()) {
        delete m_resources.begin()->second.first;
        m_resources.erase(m_resources.begin());
    }
}
```

This is a fairly straightforward method. It loops until there are no more elements in the container. Each time it deletes the resource memory and erases the container entry by passing in an iterator.

Because of the unique nature of some resources, certain methods aren't universal. For the purposes of expanding this base class to support any resource we want, a Load method is going to be used in each derived manager. In order to avoid run-time polymorphism, a **Curiously Recurring Template Pattern** can be used like so:

```
T* Load(const std::string& l_path) {
    return static_cast<Derived*>(this)->Load(l_path);
}
```

Derived classes will implement their own versions of Load, but will not rely on resolving virtual pointers to functions during run-time.

Now that the scratching of the surface is done, let's dive deep into the private methods that make this functionality possible, beginning with Find:

```
std::pair<T*, unsigned int>* Find(const std::string& l_id) {
    auto itr = m_resources.find(l_id);
    return (itr != m_resources.end() ? &itr->second : nullptr);
}
```

This method returns a pointer to a pair structure that includes the actual resource and the number of instances using it. If the string handle provided as an argument isn't located in the resource container, nullptr is returned instead.

Unloading a resource doesn't bring anything new to the table:

```
bool Unload(const std::string& l_id) {
    auto itr = m_resources.find(l_id);
    if (itr == m_resources.end()) { return false; }
    delete itr->second.first;
    m_resources.erase(itr);
    return true;
}
```

As always, we first look for the element in the container by the string handle. If it is found, we free the allocated memory, erase the element from the container and return from the method.

Lastly, we can't use those string handles without having paths that they map to. Let's load them in:

```
void LoadPaths(const std::string& l_pathFile){
    std::ifstream paths;
    paths.open(Utils::GetWorkingDirectory() + l_pathFile);
    if(paths.is_open()){
        std::string line;
        while(std::getline(paths,line)){
            std::stringstream keystream(line);
            std::string pathName;
            std::string path;
            keystream >> pathName;
            keystream >> path;
            m_paths.emplace(pathName,path);
        }
        paths.close();
        return;
    }
    std::cerr <<
        "! Failed loading the path file: "
        << l_pathFile << std::endl;
}
```

If you know anything about loading in files in C++, this should raise no eyebrows. All it does is set up an input stream called `paths`. It then tries to open it, by passing in the full path to the file, thanks to our `GetWorkingDirectory` function that was mentioned earlier. If the file is open, it means it was found and can be read. A string type is defined for use as a way of holding the current line of the file as it reads them one by one sequentially. The method loops while there's still a new line in the file we're parsing and passes in that new line to the `line` variable. A `stringstream` variable is then set up, which is designed for operations with strings. Two string variables are defined, one for the path identifier and one for the actual path. They get filled from the `keystream` variable by using its overloaded `>>` operator, which essentially just grabs everything in the line until it encounters a space delimiter. We then insert this information into the path container and close the file after the loop is over.

Implementing the texture manager

With the resource management part done, we can now implement the actual texture loading in its own class. Because there's only one method that we want to implement, it might as well be done in the header file:

```
class TextureManager:
    public ResourceManager<TextureManager, sf::Texture>
{
public:
    TextureManager(): ResourceManager("textures.cfg") {}

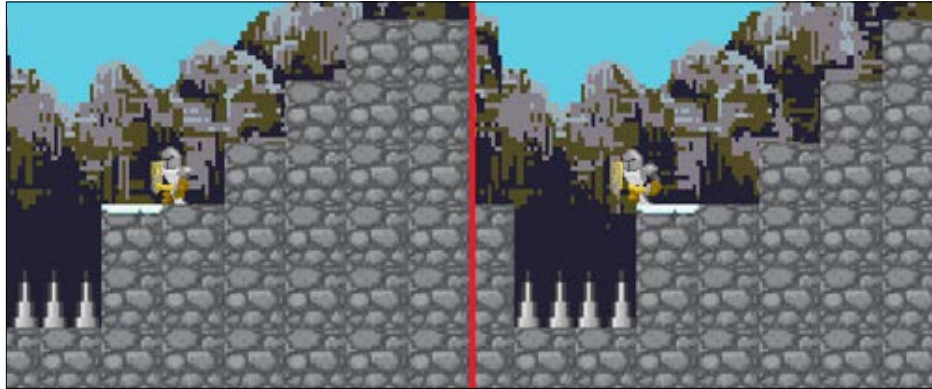
    sf::Texture* Load(const std::string& l_path) {
        sf::Texture* texture = new sf::Texture();
        if(!texture->loadFromFile(
            Utils::GetWorkingDirectory() + l_path))
        {
            delete texture;
            texture = nullptr;
            std::cerr << "! Failed to load texture: "
                << l_path << std::endl;
        }
        return texture;
    }
};
```

We create the `TextureManager` class and inherit from `ResourceManager` in addition to specifying the data types of this manager class and the resource it deals with for the template, which is, of course, a `sf::Texture`. The constructor of the texture manager is only used to call the base class constructor in the initializer list to pass in the file name that contains the bindings of handles and paths.

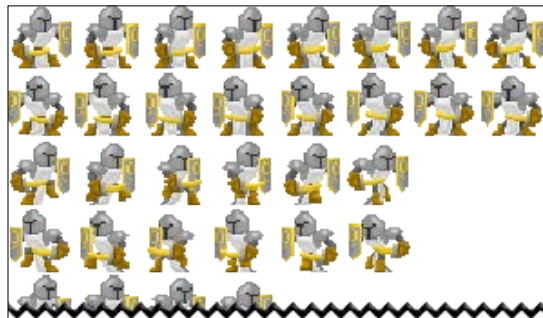
In the `Load` method, we allocate new memory for the texture and attempt to load it from the path provided as an argument. If the loading fails, we delete the allocated memory and print out a console message to notify the user of the failure. That is literally all there is to the texture manager class. Time to put it to work!

Introducing sprite sheets

First, let's whet your appetite by looking into the future of using sprite sheets, which allow you to create animations that look like this:



From our previous experience with SFML, we know that a sprite is essentially an image that can be moved around, cropped, scaled, and rotated just to mention a few options. A sprite sheet, on the other hand, is a texture that contains multiple sprites. From the image above, you can see the player is moving left and his animation is progressing. Each frame of the player's animation is stored in a sprite sheet, which is being accessed and cropped in order to represent a single sprite. This is what a small part of it looks like as a texture:



The way sprites are laid out can differ from game to game. It depends on the size constraints of a particular project, as well as the specifics of game-play. The format of the sprite sheet above is just what works the best here and is by no means the "perfect design."

Why would we want to use a sprite sheet? Well, the biggest advantage is that it makes accessing sprites easier and faster, not to mention less memory-consuming. Animation is also easier to pull off on a sprite sheet. Loading multiple textures of different sprites is more expensive than loading a single texture that can be cropped. In certain cases, careful packing of a sprite sheet can save a lot of resources. If efficiency is what you're after, use of sprite sheets over different textures for each sprite definitely takes the cake.

Implementing a sprite sheet class

Because we want to be able to modify anything related to sprite sheets on the fly, loading them from a file makes the most sense. Let's start by taking a look at what the sprite sheet for a player might look like by making a `Player.sheet` file:

```
Texture PlayerSprite
Size 32 32
Scale 1.0 1.0
|Type|Name|StartFrame|EndFrame|Row|FrameTime|FrameActionStart|End|
AnimationType Directional
Animation Idle 0 7 0 0.2 -1 -1
Animation Walk 0 5 2 0.1 -1 -1
Animation Jump 0 3 4 0.2 -1 -1
Animation Attack 0 4 6 0.08 2 3
Animation Hurt 0 2 8 0.2 -1 -1
Animation Death 0 8 10 0.15 -1 -1
```

It starts by specifying a handle of the texture that will be used. Some additional data about the sprite itself is also defined, such as the individual sprite size and scale. We then jump to a row that's commented out. It describes the order and meaning of values for the rest of the file, which is designated for defining sequences of animations in the sprite sheet. After defining the animation type, it proceeds in defining all of these different parameters about the animation. There's no need to focus on this part for now, as it will be covered in depth later.

With the file format out of the way, let's get started with the sprite sheet class! First, a container type is defined to hold the animations. An unordered map is used because it offers faster lookups than its ordered counter-part:

```
using Animations = std::unordered_map<std::string, Anim_Base*>;
```

Once again, try not to dwell on this too much, as it will be explained in more depth later. Let's write the header for the sprite sheet class:

```
class SpriteSheet{
public:
    SpriteSheet(TextureManager* l_textMgr);
    ~SpriteSheet();

    void CropSprite(const sf::IntRect& l_rect);
    ... // Basic setters/getters.
    bool LoadSheet(const std::string& l_file);
    void ReleaseSheet();

    Anim_Base* GetCurrentAnim();
    bool SetAnimation(const std::string& l_name,
        const bool& l_play = false,
        const bool& l_loop = false);

    void Update(const float& l_dT);
    void Draw(sf::RenderWindow* l_wnd);
private:
    std::string m_texture;
    sf::Sprite m_sprite;
    sf::Vector2i m_spriteSize;
    sf::Vector2f m_spriteScale;
    Direction m_direction;
    std::string m_animType;
    Animations m_animations;
    Anim_Base* m_animationCurrent;
    TextureManager* m_textureManager;
};
```

As you can see, it offers methods for cropping the texture and updating and drawing the sprite sheet. The class holds on to a pointer to a texture manager in order to obtain and free up resources. One last thing about this class is that it holds a data member of type `Direction`. It's simply just an enumeration, defined in the `Directions.h` file:

```
enum class Direction{ Right = 0, Left };
```

It's barely even worth its own header. However, quite a few classes actually rely on this, so a separate header is where it needs to be.

Let's begin implementing the actual methods of the sprite sheet class, starting, as always, with the constructor and destructor:

```
SpriteSheet::SpriteSheet(TextureManager* l_textMgr)
:m_textureManager(l_textMgr), m_animationCurrent(nullptr),
 m_spriteScale(1.f, 1.f), m_direction(Direction::Right){}
```

Apart from initializing its data members to default values, there's nothing else of interest here. The destructor simply calls another method to clean up, much like a lot of other classes:

```
SpriteSheet::~SpriteSheet() { ReleaseSheet(); }

void SpriteSheet::ReleaseSheet() {
    m_textureManager->ReleaseResource(m_texture);
    m_animationCurrent = nullptr;
    while(m_animations.begin() != m_animations.end()){
        delete m_animations.begin()->second;
        m_animations.erase(m_animations.begin());
    }
}
```

The `ReleaseSheet` method uses the texture manager to release the resource it was using, as well as delete all of the animations it has currently allocated.

When setting the sprite size, it's important to also reset the origin, so it's always in the middle of the sprite on the *x* axis and down all the way on the *y* axis:

```
void SpriteSheet::SetSpriteSize(const sf::Vector2i& l_size) {
    m_spriteSize = l_size;
    m_sprite.setOrigin(m_spriteSize.x / 2, m_spriteSize.y);
}
```

Naturally, we also need a method for setting the position of the sprite:

```
void SpriteSheet::SetSpritePosition(const sf::Vector2f& l_pos) {
    m_sprite.setPosition(l_pos);
}
```

Setting a different direction of a sprite will change its sprite, so we need to re-crop it afterwards:

```
void SpriteSheet::SetDirection(const Direction& l_dir) {
    if (l_dir == m_direction) { return; }
    m_direction = l_dir;
    m_animationCurrent->CropSprite();
}
```

The actual cropping is done through the `setTextureRect` method of a sprite class:

```
void SpriteSheet::CropSprite(const sf::IntRect& l_rect) {
    m_sprite.setTextureRect(l_rect);
}
```

It takes in a `sf::IntRect` type, which defines its top-left corner, as well as the size of the rectangle. The top-left corner coordinates are local to the texture that's being cropped. Let's say we want to obtain the first sprite in the sprite sheet. If we know that each sprite is 32px by 32px in size, all we need to do is pass in the position (0;0) for the top-left corner and then the size (32;32) in order to obtain the sprite.

Although we haven't covered animations yet, let's get the `SetAnimation` method out of the way, since it's not too difficult to understand, even without knowing every specific detail about our soon-to-be animation class:

```
bool SpriteSheet::SetAnimation(const std::string& l_name,
    const bool& l_play, const bool& l_loop)
{
    auto itr = m_animations.find(l_name);
    if (itr == m_animations.end()) { return false; }
    if (itr->second == m_animationCurrent) { return false; }
    if (m_animationCurrent) { m_animationCurrent->Stop(); }
    m_animationCurrent = itr->second;
    m_animationCurrent->SetLooping(l_loop);
    if(l_play) { m_animationCurrent->Play(); }
    m_animationCurrent->CropSprite();
    return true;
}
```

It takes in three arguments: a string handle and two Boolean flags for playing the animation instantly and whether it's looped or not. The method itself looks through the animation container for an animation that matches the string handle. If one is found, it checks if any current animation is set because it needs to be stopped. Once that is done, it simply changes the pointer to the current animation to the animation that was found in the container, as well as calls the respective methods in order to loop and play the animation. Nothing too complicated.

We then top off this class with the most run of the mill update and draw methods:

```
void SpriteSheet::Update(const float& l_dT) {
    m_animationCurrent->Update(l_dT);
}

void SpriteSheet::Draw(sf::RenderWindow* l_wnd) {
    l_wnd->draw(m_sprite);
}
```

This is about as simple as it gets. However, it does leave us with one method unaccounted for: `LoadSheet`. Before we can implement that, we need to know more about the animation classes we'll be working with.

The base animation class

Much like with the resource manager, we want to off-load any and all functionality that isn't unique to more specific classes and put it in a base class. This is where the base animation class comes in. Let's take a look at the `Anim_Base.h` header file:

```
class SpriteSheet;
using Frame = unsigned int;

class Anim_Base{
    friend class SpriteSheet;
public:
    Anim_Base();
    virtual ~Anim_Base();
    ... // Setters/getters.
    void Play();
    void Pause();
    void Stop();
    void Reset();

    virtual void Update(const float& l_dT);

    friend std::stringstream& operator >>(
        std::stringstream& l_stream, Anim_Base& a)
    {
        a.ReadIn(l_stream);
        return l_stream;
    }
protected:
    virtual void FrameStep() = 0;
    virtual void CropSprite() = 0;
    virtual void ReadIn(std::stringstream& l_stream) = 0;

    Frame m_frameCurrent;
    Frame m_frameStart;
    Frame m_frameEnd;
    Frame m_frameRow;
    int m_frameActionStart; // Frame when a specific "action" begins
    int m_frameActionEnd; // Frame when a specific "action" ends
    float m_frameTime;
```

```

float m_elapsedTime;
bool m_loop;
bool m_playing;
std::string m_name;
SpriteSheet* m_spriteSheet;
};

```

First, notice the forward declaration of class `SpriteSheet`. Because this class needs to include `SpriteSheet` and `SpriteSheet` needs to include this class, forward declarations are necessary to prevent cross-inclusions. We're also going to be using an alias for the unsigned integer type, simply named `Frame`.

Most of the data member as well as method names are fairly self-explanatory. A few terms might be confusing, such as frame time and action. Frame time is the amount of time each frame takes to finish. Action defines a range of frames during which a behavior which is specific to that animation can be performed. If it's set to negative one, this behavior can be performed throughout the entire animation. These are some things that we want to keep track of in order to make the game more interactive and responsive. Note that we're overloading the `>>` operator in order to ease animation loading from files. More on that later.

The last thing to point out is the three purely virtual methods: `FrameStep`, `CropSprite`, and `ReadIn`. `FrameStep` is the update portion that's unique to different types of animations. `CropSprite` is a unique way different types of animations would obtain sprites from a sprite sheet. Lastly, `ReadIn` is the method that defines how exactly the stringstream object is used when data is being loaded from files. These three methods will only be defined in derived classes.

Implementing the base animation class

Due to forward declarations, we need to include the actual header files of classes that were declared in the `.cpp` file:

```

#include "Anim_Base.h"
#include "SpriteSheet.h"

```

Now we have no more cross-inclusions and we get to use the `SpriteSheet` class. Time to implement the actual class:

```

Anim_Base::Anim_Base(): m_frameCurrent(0), m_frameStart(0),
    m_frameEnd(0), m_frameRow(0), m_frameTime(0.f),
    m_elapsedTime(0.f), m_frameActionStart(-1),
    m_frameActionEnd(-1), m_loop(false), m_playing(false){}
Anim_Base::~Anim_Base(){}

```

The constructor is doing its intended job of initializing default values, while the destructor isn't going to be used at all in this class.

Of course, we need a method for setting our sprite sheet data member to something:

```
void Anim_Base::SetSpriteSheet(SpriteSheet* l_sheet){
    m_spriteSheet = l_sheet;
}
```

The same thing is true for setting the animation frame, although this method is slightly more intricate:

```
void Anim_Base::SetFrame(const unsigned int& l_frame){
    if((l_frame >= m_frameStart && l_frame <= m_frameEnd) ||
        (l_frame >= m_frameEnd && l_frame <= m_frameStart))
    {
        m_frameCurrent = l_frame;
    }
}
```

The argument that is passed to this method is checked for being in two specific ranges, which is done in order to add support for types of animation that can play backwards in the future.

Here's a method for checking if this animation is currently able to perform its custom behavior:

```
bool Anim_Base::IsInAction(){
    if(m_frameActionStart == -1 || m_frameActionEnd == -1){
        return true;
    }

    return (m_frameCurrent >= m_frameActionStart
        && m_frameCurrent <= m_frameActionEnd);
}
```

If any of the values are -1, the "action" is always performed. Otherwise, the current frame is checked for being within the designated range that will be loaded from the sprite sheet file.

We can't go far without controlling these animations in one way or another. It's a good idea to provide a simple interface to do that:

```
void Anim_Base::Play(){ m_playing = true; }
void Anim_Base::Pause(){ m_playing = false; }
void Anim_Base::Stop(){ m_playing = false; Reset(); }
```

The `Play` and `Pause` methods simply manipulate a Boolean flag, while the `Stop` method also resets the animation:

```
void Anim_Base::Reset(){
    m_frameCurrent = m_frameStart;
    m_elapsedTime = 0.0f;
    CropSprite();
}
```

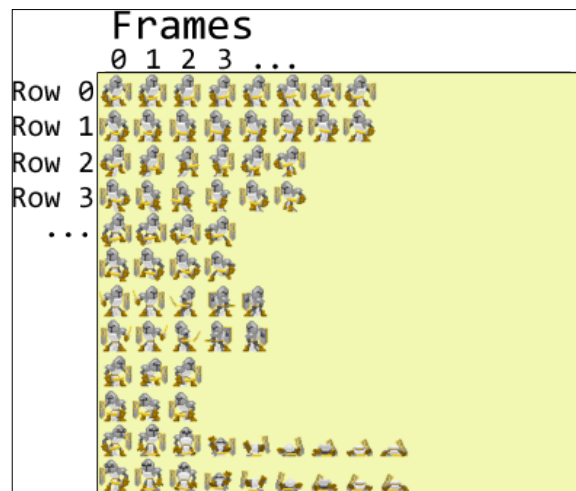
After moving the frame back to the beginning and resetting the timer, it crops the sprite because of the frame change. We are almost done with this class. All that's missing now is a way to update it:

```
void Anim_Base::Update(const float& l_dT){
    if (!m_playing){ return; }
    m_elapsedTime += l_dT;
    if (m_elapsedTime < m_frameTime){ return; }
    FrameStep();
    CropSprite();
    m_elapsedTime = 0;
}
```

The `Update` method, as per usual, takes in an argument that represents the elapsed time between frames. It then simply adds it to the elapsed time of the animation if it's playing and checks if it has exceeded the frame time. If it did, our two virtual methods are called and the timer is reset back to 0.

Directional animation

There's not always a clear dichotomy present between different types of animation depending on implementation details. For the purposes of this chapter not dragging on with specific subjects, only one type of animation will be implemented, which is the directional animation. This type of animation is usually used with any kind of moving entities that have a specific animation for each direction. Unlike other types of animation, where an increased frame can lead to a jump in rows, directional animation will always remain on the row that represents the proper type of animation in the proper direction. Consider the following illustration:



In our case, each row contains either the left or the right facing version of a specific animation. Knowing that, let's create the header of the directional animation class:

```
class Anim_Directional : public Anim_Base{
protected:
    void FrameStep();
    void CropSprite();
    void ReadIn(std::stringstream& l_stream);
};
```

This class doesn't even require a constructor or a destructor, only the three methods from the base class that need to be implemented. Once again, note the inclusion of the `SpriteSheet.h` file due to its forward declaration in the header of the `Anim_Base` class:

```
#include "Anim_Directional.h"
#include "SpriteSheet.h"
```

Now let's slice through that texture to fashion our sprite:

```
void Anim_Directional::CropSprite() {
    sf::IntRect rect(
        m_spriteSheet->GetSpriteSize().x * m_frameCurrent,
        m_spriteSheet->GetSpriteSize().y *
            (m_frameRow + (short)m_spriteSheet->GetDirection()),
        m_spriteSheet->GetSpriteSize().x,
        m_spriteSheet->GetSpriteSize().y);
    m_spriteSheet->CropSprite(rect);
}
```

First, we construct a rectangle. Its top-left corner position is the sprite size multiplied by the current frame on the x axis and the sprite size multiplied by the sum of the current animation row and the sprite sheet direction on the y axis. Because the direction enumeration maps the directions to numerical values of either 0 or 1, it makes obtaining the row for the correct direction really easy, as shown above. After the top-left corner is set up, we pass in the sprite size in pixels and crop the sprite sheet based on the rectangle that was constructed. This yields us a sprite!

The final piece of the puzzle in the animation department is implementing the `FrameStep` method:

```
void Anim_Directional::FrameStep() {
    if (m_frameStart < m_frameEnd) { ++m_frameCurrent; }
    else { --m_frameCurrent; }

    if ((m_frameStart < m_frameEnd && m_frameCurrent > m_frameEnd) ||
        (m_frameStart > m_frameEnd && m_frameCurrent < m_frameEnd))
    {
        if (m_loop) { m_frameCurrent = m_frameStart; return; }
        m_frameCurrent = m_frameEnd;
        Pause();
    }
}
```

First, we check which direction we should roll the frames in, since it could be necessary in the future to define backwards-moving animations. If the starting frame number is lower than the ending frame number, we're moving in the positive direction. We then check if the frame is out of bounds, and based on whether it is looping or not, we either reset the current frame to start, or set it to the end of the animation and then pause it. The same logic applies if the animation is playing backwards, except the direction is reversed.

Lastly, the method responsible for reading in data from files:

```
void Anim_Directional::ReadIn(std::stringstream& l_stream){
    l_stream >> m_frameStart >> m_frameEnd >> m_frameRow
        >> m_frameTime >> m_frameActionStart >> m_frameActionEnd;
}
```

With that final bit of code, the animation portion is finished! Everything we need in order to implement loading the sprite sheet file is now available.

Loading the sprite sheet files

The loading method begins, as per usual, by setting up the file, reading it, and obtaining the current line. The first identifier from the line is loaded into the type variable. The rest is fairly typical:

```
bool SpriteSheet::LoadSheet(const std::string& l_file){
    std::ifstream sheet;
    sheet.open(Utils::GetWorkingDirectory() + l_file);
    if(sheet.is_open()){
        ReleaseSheet(); // Release current sheet resources.
        std::string line;
        while(std::getline(sheet,line)){
            if (line[0] == '|'){ continue; }
            std::stringstream keystream(line);
            std::string type;
            keystream >> type;
            ...
        }
        sheet.close();
        return true;
    }
    std::cerr << "! Failed loading spritesheet: "
        << l_file << std::endl;
    return false;
}
```

In order to avoid confusion, the parsing of different types of entries in this file has been split up into separate sections. Let's begin with the texture loading:

```
if(type == "Texture"){
    if (m_texture != ""){
        std::cerr << "! Duplicate texture entries in: "
            << l_file << std::endl;
        continue;
    }
}
```

```

    }
    std::string texture;
    keystream >> texture;
    if (!m_textureManager->RequireResource(texture)) {
        std::cerr << "! Could not set up the texture: "
            << texture << std::endl;
        continue;
    }
    m_texture = texture;
    m_sprite.setTexture(*m_textureManager->GetResource(m_texture));
} else if ...

```

First, we check if the texture hasn't been initialized already in order to avoid duplicate entries. If it hasn't, the `keystream` variable spits out the texture handle, which gets passed into the texture manager in an `if` statement. This is done to catch errors of invalid handles. If the handle is valid, the texture name is kept around for later freeing of resources and the sprite we'll be using for drawing is set to point to the texture.

Time to read the smaller bits of information in:

```

    } else if (type == "Size") {
        keystream >> m_spriteSize.x >> m_spriteSize.y;
        SetSpriteSize(m_spriteSize);
    } else if (type == "Scale") {
        keystream >> m_spriteScale.x >> m_spriteScale.y;
        m_sprite.setScale(m_spriteScale);
    } else if (type == "AnimationType") {
        keystream >> m_animType;
    } else if ...

```

The most dramatic entry has been saved for last. At this moment, we parse the animations:

```

    } else if (type == "Animation") {
        std::string name;
        keystream >> name;
        if (m_animations.find(name) != m_animations.end()) {
            std::cerr << "! Duplicate animation(" << name
                << ") in: " << l_file << std::endl;
            continue;
        }
        Anim_Base* anim = nullptr;
        if (m_animType == "Directional") {
            anim = new Anim_Directional();
        } else {

```

```
        std::cerr << "! Unknown animation type: "
        << m_animType << std::endl;
        continue;
    }

    keystoream >> *anim;
    anim->SetSpriteSheet(this);
    anim->SetName(name);
    anim->Reset();
    m_animations.emplace(name, anim);

    if (m_animationCurrent){ continue; }
    m_animationCurrent = anim;
    m_animationCurrent->Play();
}
```

First, the animation name gets loaded and the animation container is checked in order to avoid duplicates. The type of animation that was loaded previously is then checked in order to construct a correct animation type. We could use a factory method for this, but since we only have one type of animation so far, it seems pointless at this time. The animation structure then gets data streamed into it from our `stringstream` object, initializing it. Furthermore, the animation is reset in order to zero-out its values. Once it gets inserted into the animation container, the last thing we check for is whether the current animation member has been assigned a value yet. If it hasn't, this is the first animation in the sprite sheet file, which we're assuming is the default. It gets assigned to the current animation member and set to play.

Summary

While having good graphics isn't the most important aspect of a game, going from basic shapes to actual sprites being animated on screen can make a world of difference in the eyes of a player. Granted, prettifying a product doesn't fix whatever flaws it may have, which seems like a popular mentality nowadays. Immersing the player into the game world, however, as well as bringing to life what seemed like a bunch of squares, is the effect we're after, which, with the completion of this chapter, you are now able to achieve with a few basic techniques.

In the next chapter, we will be covering common game development elements that will unify all of the graphical bits and pieces we built into a fully functional game with platforming elements, enemies, and multiple levels. See you there!